

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



DSA-Extended

Báo cáo bài tập lớn (Mở rộng)
PII-NER-ANONYMIZER

GVHD: Vương Bá Thịnh

Sinh viên thực hiện:	Trần Đình Ý	2414100
	Trần Văn Minh Triết	2413603
	Nguyễn Phúc Vĩnh	2414001
	Nguyễn Đình Minh	2412079

HỒ CHÍ MINH, THÁNG 5/ 2026

Mục lục

Mục lục	i
Danh sách hình vẽ	ii
Danh sách bảng	iii
Danh sách đoạn mã	iv
1 Giới thiệu đề tài	1
2 Tổng quan mô hình và pipeline của hệ thống	2
3 Thiết kế module	3
3.1 Nhận diện thực thể theo ngữ cảnh (Context Detector)	3
3.1.1 Nhận diện tên người, tổ chức và địa điểm	3
3.1.2 Nhận diện địa chỉ	6
3.2 Tổng hợp kết quả	8
3.2.1 Vấn đề overlap	9
3.2.2 Hệ thống ưu tiên	9
3.2.3 Thuật toán Greedy Priority-First	9
3.2.4 Minh họa trường hợp overlap	10

Danh sách hình vẽ

2.1	Pipeline tổng quát của hệ thống	3
-----	---	---

Danh sách bảng

3.1	Danh sách thực thể trước khi hợp nhất	10
3.2	Danh sách thực thể sau khi hợp nhất	11

Danh sách đoạn mã

PHÂN CÔNG NHIỆM VỤ VÀ KẾT QUẢ CỦA TỪNG THÀNH VIÊN

STT	HỌ VÀ TÊN	MSSV	NHIỆM VỤ	KẾT QUẢ
1	Trần Văn Minh Triết	2413603	None.	100%
2	Nguyễn Đình Minh	2412079	None.	100%
3	Nguyễn Phúc Vĩnh	2414001	None.	100%
4	Trần Đình Ý	2414100	None.	100%

1 Giới thiệu đề tài

Named Entity Recognition, thường được viết tắt là NER, là một bài toán quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên. Mục tiêu chính của NER là phát hiện và phân loại các thực thể có ý nghĩa trong văn bản, chẳng hạn như tên người, địa điểm, tổ chức, email, số điện thoại, địa chỉ, tên tài khoản hoặc đường dẫn URL. Thay vì chỉ xem văn bản như một chuỗi ký tự thông thường, NER giúp hệ thống hiểu được những thành phần nào trong văn bản mang thông tin quan trọng và chúng thuộc loại nào.

Trong thực tế, NER được ứng dụng trong nhiều bài toán khác nhau. Ví dụ, trong hệ thống tìm kiếm thông tin, NER giúp trích xuất tên người, địa danh hoặc tổ chức để cải thiện độ chính xác khi truy vấn. Trong lĩnh vực chăm sóc khách hàng, NER có thể dùng để nhận diện thông tin đơn hàng, địa chỉ giao hàng hoặc số điện thoại của người dùng. Trong các hệ thống quản lý tài liệu, NER hỗ trợ tự động phân loại và trích xuất thông tin quan trọng từ văn bản. Đặc biệt, đối với bài toán bảo vệ dữ liệu cá nhân, NER có vai trò phát hiện các thông tin định danh cá nhân, từ đó phục vụ cho việc ẩn danh hoặc che giấu dữ liệu nhạy cảm trước khi lưu trữ, chia sẻ hoặc xử lý tiếp.

Trong phạm vi project này, bài toán được tập trung vào việc nhận diện và ẩn danh thông tin cá nhân trong văn bản. Các loại thông tin cần phát hiện bao gồm **NAME**, **EMAIL**, **PHONE**, **ADDRESS**, **USERNAME**, **ORGANIZATION**, **LOCATION** và **URL**. Đây là những dạng dữ liệu có khả năng định danh trực tiếp hoặc gián tiếp một cá nhân. Ví dụ, một đoạn văn bản như:

My name is John Smith and my email is john@gmail.com.

sau khi xử lý có thể được chuyển thành:

My name is [NAME] and my email is [EMAIL].

Cách xử lý này giúp giảm nguy cơ lộ thông tin cá nhân, đồng thời vẫn giữ lại cấu trúc tổng quát của văn bản để phục vụ cho các bước phân tích tiếp theo.

Có nhiều hướng tiếp cận khác nhau để giải quyết bài toán NER. Hướng thứ nhất là sử dụng các phương pháp học máy hoặc học sâu, trong đó mô hình được huấn luyện trên dữ liệu đã gán nhãn để tự học cách nhận diện thực thể. Các mô hình hiện đại có thể đạt độ chính xác cao, đặc biệt khi có lượng dữ liệu lớn và tài nguyên tính toán phù hợp. Tuy nhiên, hướng tiếp cận này thường yêu cầu quá trình huấn luyện phức tạp, phụ thuộc nhiều vào dữ liệu, và khó giải thích rõ từng quyết định của hệ thống.

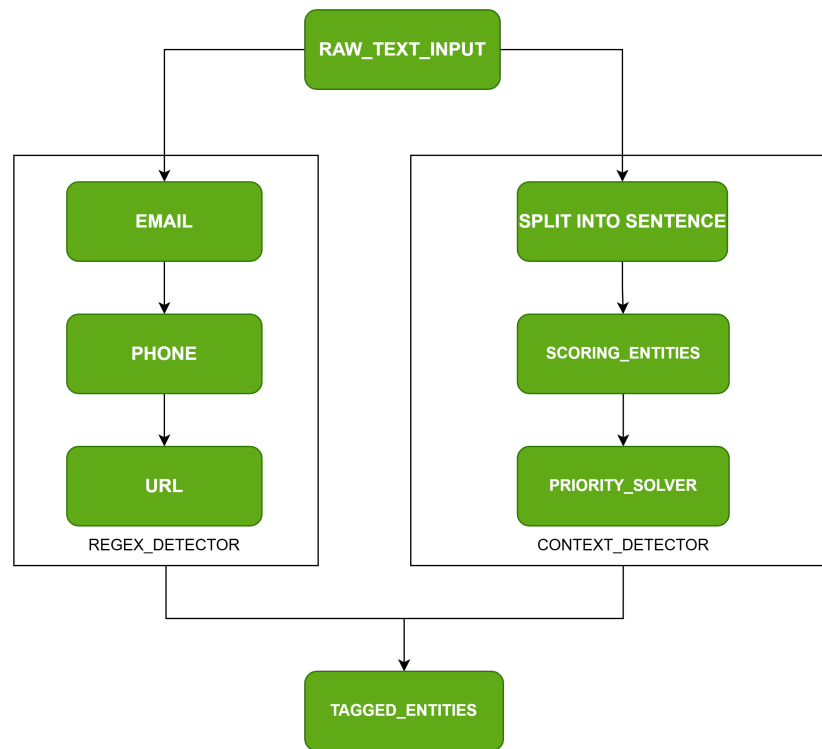
Hướng thứ hai là sử dụng phương pháp rule-based, tức là xây dựng các luật thủ công dựa trên đặc điểm chuỗi, biểu thức chính quy và ngữ cảnh xuất hiện của thực thể. Ví

dụ, email thường có ký tự @ và tên miền phía sau, URL thường bắt đầu bằng `http://`, `https://` hoặc `www.`, còn số điện thoại thường là chuỗi chữ số có độ dài nhất định và có thể chứa dấu cách, dấu gạch ngang hoặc mã quốc gia. Đối với các thực thể khó hơn như tên người hoặc địa chỉ, hệ thống có thể dựa vào các từ khóa ngữ cảnh như `my name is`, `full name`, `address`, `street`, `road`, `city`, v.v.

Trong project này, nhóm lựa chọn hướng rule-based thay vì machine learning hoặc deep learning. Lý do là project tập trung vào việc áp dụng các thuật toán xử lý chuỗi, regular expression, keyword matching, token scanning và xử lý khoảng giao nhau giữa các entity. Cách tiếp cận này phù hợp với mục tiêu học thuật vì dễ hiện thực, dễ kiểm soát, dễ giải thích và không cần quá trình huấn luyện mô hình. Ngoài ra, rule-based method cũng cho phép từng thành viên trong nhóm phát triển các module độc lập, sau đó tích hợp lại thành một pipeline hoàn chỉnh.

2 Tổng quan mô hình và pipeline của hệ thống

Pipeline tổng quát của hệ thống gồm các bước chính. Đầu tiên, dữ liệu thô được đọc từ file CSV và xử lý ở bước preprocessing. Trong bước này, các token và nhãn BIO trong dataset được chuyển thành danh sách ground truth entities để phục vụ đánh giá. Tiếp theo, hệ thống sử dụng regex-based detector để nhận diện các thực thể có cấu trúc rõ ràng như email, số điện thoại và URL. Sau đó, context-based detector được dùng để nhận diện các thực thể phụ thuộc nhiều vào ngữ cảnh như tên người, tổ chức, địa điểm, địa chỉ và username. Kết quả từ hai detector được đưa vào module merger để gộp, loại bỏ trùng lặp và xử lý các entity bị chồng lấn. Cuối cùng, hệ thống thực hiện ẩn danh văn bản và đánh giá kết quả bằng các chỉ số Precision, Recall và F1-score.



Hình 2.1: Pipeline tổng quát của hệ thống

3 Thiết kế module

3.1 Nhận diện thực thể theo ngữ cảnh (Context Detector)

Thành phần nhận diện theo ngữ cảnh nhận đầu vào là một đoạn văn bản thô và trả về danh sách các thực thể có dạng `type`, `text`, `start`, `end`. Khác với nhóm nhận diện bằng biểu thức chính quy, nhóm này không chỉ dựa vào cấu trúc ký tự cố định mà còn khai thác các từ khóa, vị trí xuất hiện, chữ hoa và quan hệ giữa các từ trong cùng câu.

Các loại thực thể được xử lý gồm tên người (`NAME`), tổ chức (`ORGANIZATION`), địa điểm (`LOCATION`), tên người dùng (`USERNAME`) và địa chỉ (`ADDRESS`). Về mặt thuật toán, quá trình nhận diện được tổ chức theo luồng: chuẩn hóa ngữ cảnh đầu vào, sinh candidate, chấm điểm hoặc kiểm tra mẫu, sau đó trả về các span thực thể không chồng lấn.

3.1.1 Nhận diện tên người, tổ chức và địa điểm

Đầu vào của thuật toán là văn bản đã được chia thành các câu cùng với vị trí của từng câu trong văn bản gốc. Đầu ra là các span được gán một trong ba nhãn: `NAME`,

ORGANIZATION hoặc LOCATION. Ba loại thực thể này đều có thể xuất hiện dưới dạng cụm danh từ riêng viết hoa, vì vậy hệ thống không chạy ba bộ phát hiện tách rời. Thay vào đó, thuật toán sinh candidate một lần, chấm điểm candidate đó theo cả ba nhãn, rồi quyết định nhãn cuối cùng ở bước sau cùng.

Trước khi phân tích ngữ cảnh, văn bản đầu vào được tách thành các câu riêng lẻ. Đây là bước quan trọng vì điểm ngữ cảnh (pronoun, nghề nghiệp, trigger) chỉ có ý nghĩa trong phạm vi một câu.

Thách thức chính là phân biệt dấu chấm kết thúc câu với dấu chấm trong các trường hợp khác. Thuật toán tách câu xử lý điều này theo quy tắc: dấu . chỉ được coi là ranh giới câu khi *đồng thời* thỏa ba điều kiện:

1. Token kết thúc bằng . không thuộc danh sách viết tắt đã biết (Dr., Mr., St., Apt., Inc., v.v.).
 2. Phần văn bản ngay sau . không bắt đầu bằng tên miền cấp cao (com, edu, vn, org, v.v.) — nhằm tránh split nhầm địa chỉ email và URL.
 3. Sau dấu . là khoảng trắng hoặc hết chuỗi.
- Dấu ! và ? luôn được coi là kết thúc câu vô điều kiện.

Bước 1: xử lý trigger mạnh cho tên người

Thuật toán trước hết tìm các cụm từ báo hiệu trực tiếp rằng phần đứng sau là tên người. Các trigger này không tạo ra một luồng nhận diện riêng, mà được lưu như một tín hiệu điểm mạnh cho nhãn NAME. Trigger được chia thành hai nhóm:

- **NAME_TRIGGER**: cụm từ chỉ thẳng tên chủ thể, gồm "my name is", "full name", "name:".
- **_LABEL_TRIGGER**: nhãn hay đứng trước tên trong văn bản có cấu trúc, gồm khoảng 25 mẫu như "contact information:", "author:", "submitted by:", "regards,", "sincerely,", v.v.

Khi tìm thấy trigger, thuật toán thu thập 1–4 token liên tiếp thỏa: bắt đầu bằng chữ hoa, không chứa chữ số, không chứa ký tự đặc biệt (ngoại trừ dấu gạch ngang – vì tên có thể có dạng Mary-Jane), không phải stopword, và không mang đuôi động từ sau token đầu tiên (-ing, -tion, -ed, v.v.). Span tìm được sẽ nhận điểm NAME rất cao nếu trùng với candidate ở bước chấm điểm.

Bước 2: tìm candidate bằng linear search và rule-based filtering

Với mỗi câu, thuật toán sinh các candidate danh từ riêng từ những cụm token viết hoa. Candidate được dùng chung cho cả ba nhãn NAME, ORGANIZATION và LOCATION, thay

vì viết lại ba hàm quét riêng.

Một candidate hợp lệ phải bắt đầu bằng chữ hoa, không chứa chữ số, không chứa ký tự đặc biệt, không phải stopword, không phải đại từ, không phải từ mở đầu câu thường gặp, không phải title prefix và không phải chức danh công việc. Thuật toán cũng cho phép một số connector như *of*, *the*, *and* để bắt các cụm như *University of California*.

Bước 3: scoring candidate theo ba nhãn

Chương trình sử dụng thuật toán Heuristic Rule-Based Scoring để gán điểm cho mỗi candidate theo ba nhãn. Mỗi nhãn có một tập hợp các quy tắc riêng. Ví dụ: tần suất xuất hiện của các từ đặc biệt, cấu trúc cố định,...

Output

Sau khi có đủ ba điểm, thuật toán chọn đúng một nhãn cuối cùng cho candidate. Nhãn có điểm cao nhất và đạt ngưỡng sẽ được giữ. Nếu nhiều nhãn bằng điểm, thuật toán dùng thứ tự tie-break ổn định để tránh một span được xuất ra nhiều lần với nhiều tag khác nhau. Vì vậy, một span như *OpenAI* trong câu *I work at OpenAI*. được quyết định là *ORGANIZATION*, không đồng thời xuất hiện thêm dưới dạng *NAME* hoặc *LOCATION*.

Sau khi mỗi candidate đã có nhãn cuối cùng, hệ thống loại trùng và overlap theo priority nội bộ, điểm, vị trí bắt đầu và độ dài span trong phạm vi các kết quả do context detector sinh ra. Kết quả này vẫn có thể được đưa sang merger tổng thể để xử lý overlap với các thực thể từ regex detector.

Algorithm 1 Thuật toán nhận diện NAME, ORGANIZATION và LOCATION

Require: Văn bản đầu vào T

Ensure: Danh sách thực thể E

```
1:  $E \leftarrow \emptyset$ 
2:  $lower \leftarrow \text{lowercase}(T)$ 
// Bước 1: Ghi nhận trigger mạnh cho NAME
3: for mỗi trigger  $tr$  trong  $\text{NAMETRIGGERS} \cup \text{LABELTRIGGERS}$  do
4:   for mỗi vị trí  $pos$  của  $tr$  trong  $lower$  do
5:     Thu thập 1–4 token hoa liên tiếp sau  $pos$  vào  $tokens$ 
6:     if  $tokens \neq \emptyset$  then
7:       Lưu span  $\text{join}(tokens)$  vào  $strongNameSpans$  với điểm 100
8:     end if
9:   end for
10: end for
// Bước 2–4: Candidate chung, scoring ba nhãn, chọn tag
11:  $sentences \leftarrow \text{SPLITSENTENCES}(T)$ 
12: for mỗi câu  $s$  trong  $sentences$  do
13:   for mỗi candidate  $c$  trong  $\text{EXTRACTENTITYCANDIDATES}(s)$  do
14:      $nameScore \leftarrow \text{SCORENAME}(c, s, strongNameSpans)$ 
15:      $orgScore \leftarrow \text{SCOREORGANIZATION}(c, s)$ 
16:      $locScore \leftarrow \text{SCORELOCATION}(c, s)$ 
17:      $tag \leftarrow \text{CHOOSEBESTTAG}(nameScore, orgScore, locScore)$ 
18:     if  $tag$  tồn tại then
19:       Thêm  $c$  vào  $E$  với nhãn  $tag$  và điểm tương ứng
20:     end if
21:   end for
22: end for
23: return  $\text{RESOLVEENTITIES}(E)$ 
```

3.1.2 Nhận diện địa chỉ

Đầu vào của thuật toán nhận diện địa chỉ là văn bản gốc và danh sách span email đã được xác định trước. Đầu ra là các span ADDRESS. Địa chỉ trong tập dữ liệu thường tuân theo cấu trúc gồm số nhà, tên đường, loại đường, và hậu tố hướng hoặc đơn vị căn hộ tùy chọn. Thuật toán sử dụng hai cơ chế bổ sung cho nhau.

Bước 1: nhận diện địa chỉ bằng Regex pattern

Biểu thức chính quy `_ADDRESS_RE` mô tả cấu trúc địa chỉ theo bốn thành phần nối tiếp:

1. **Số nhà:** `\b(\d{1,5})(?!\\d)\\s+` — từ 1 đến 5 chữ số, kèm negative lookahead `(?!\\d)` để đảm bảo đây không phải là một phần của số dài hơn (loại trừ số điện thoại, mã bưu điện dài).
2. **Tên đường:** `((?:[A-Z0-9][A-Za-z0-9]*\\s+)+?)` — một hoặc nhiều từ bắt đầu bằng chữ hoa hoặc chữ số, sử dụng non-greedy để nhường ưu tiên cho thành phần loại đường phía sau.
3. **Loại đường:** danh sách khoảng 30 từ được sắp xếp từ dài đến ngắn để tránh match sớm, bao gồm dạng đầy đủ (Boulevard, Terrace, Avenue, Street, Drive, Road, v.v.) và dạng viết tắt (Blvd., Ave., Rd., St., v.v.).
4. **Hướng và đơn vị** (tùy chọn): hướng gồm tám giá trị North, South, East, West và các tổ hợp; đơn vị gồm Suite, Apt., Apartment, Unit kèm số phòng.

Trước khi kiểm tra kết quả regex, tất cả span email trong văn bản được xác định trước; bất kỳ span địa chỉ nào nằm hoàn toàn bên trong span email sẽ bị bỏ qua để tránh nhầm lẫn phần tên miền của địa chỉ email với địa chỉ nhà.

Bước 2: nhận diện địa chỉ bằng trigger keyword

Để bổ sung cho regex chính, đặc biệt với các địa chỉ được viết theo dạng nhân hoặc clause theo trigger, module sử dụng danh sách trigger bao gồm **street, road, avenue, district, city, st., rd., ave..** Khi tìm thấy trigger, thuật toán chỉ tiếp tục nếu ngay sau trigger có dấu : hoặc cụm **is**. Sau đó hệ thống lấy clause đến dấu câu gần nhất, loại bỏ noise words đầu clause (**at, is, the, located, live, reside, v.v.**), và kiểm tra thêm: số nhà ở đầu clause không được vượt quá 5 chữ số, span không nằm trong email, và span không được trùng lặp với kết quả từ cơ chế regex.

Algorithm 2 Thuật toán nhận diện địa chỉ

Require: Văn bản đầu vào T

Ensure: Danh sách thực thể địa chỉ E

```
1:  $E \leftarrow \emptyset$ 
2:  $emailSpans \leftarrow \text{FINDEMAILSPANS}(T)$ 
// Cơ chế 1: Regex pattern
3: for mỗi match  $m$  của ADDRESSRE trong  $T$  do
4:   if  $m$  nằm trong một email span then continue
5:   end if
6:   Thêm span của  $m$  vào  $E$ 
7: end for
// Cơ chế 2: Trigger keyword
8: for mỗi trigger  $tr$  trong ADDRESSTRIGGERS do
9:   for mỗi vị trí  $pos$  của  $tr$  trong  $T$  do
10:    if sau  $tr$  không phải : hoặc is then continue
11:    end if
12:     $clause \leftarrow \text{EXTRACTCLAUSE}(T, pos)$ 
13:     $addr \leftarrow \text{STRIPLEADINGNOISE}(clause)$ 
14:    if số nhà đầu  $addr$  có nhiều hơn 5 chữ số then continue
15:    end if
16:    if  $addr$  nằm trong email span then continue
17:    end if
18:    if  $addr$  overlap với span đã có trong  $E$  then continue
19:    end if
20:    Thêm span của  $addr$  vào  $E$ 
21:  end for
22: end for
23: return DEDUPLICATE( $E$ )
```

3.2 Tổng hợp kết quả

Đầu vào của bước hợp nhất là nhiều danh sách thực thể được sinh ra từ các nhóm nhận diện khác nhau. Đầu ra là một danh sách duy nhất, đã loại trùng và không còn các span chồng lấn. Thuật toán hợp nhất là bước quyết định kết quả cuối cùng trước khi văn bản được mã hoá.

3.2.1 Vấn đề overlap

Hai thực thể được coi là *chồng lấn* (overlap) nếu khoảng ký tự của chúng giao nhau, tức là $[s_1, e_1)$ và $[s_2, e_2)$ thỏa $s_1 < e_2$ và $e_1 > s_2$. Overlap xảy ra trong nhiều tình huống thực tế:

- Địa chỉ email `kazuosun@hotmail.net` bị `regex_detector` bắt đúng là **EMAIL**, trong khi các detector khác có thể tạo span chồng lấn trên một phần của chuỗi này.
- Một chuỗi như `123 Main Street` có thể được nhận là **ADDRESS**, đồng thời phần `Main Street` cũng có thể được nhận là **LOCATION**.
- Một cụm như `OpenAI Research Lab` có thể được chấm điểm theo nhiều nhãn ngữ cảnh; khi đó detector cần chọn một nhãn cuối cùng trước khi đưa kết quả sang bước hợp nhất.

3.2.2 Hệ thống ưu tiên

Để giải quyết overlap một cách nhất quán, merger định nghĩa một thứ tự ưu tiên tuyến tính cho tám loại thực thể:

EMAIL > URL > PHONE > ADDRESS > ORGANIZATION > LOCATION > NAME > USERNAME

Thứ tự này phản ánh độ tin cậy của từng detector: **EMAIL**, **URL** và **PHONE** được phát hiện bằng regex có độ chính xác cao và cấu trúc rõ ràng nên được ưu tiên hơn; **ADDRESS** thắng **LOCATION** để tránh mất địa chỉ đầy đủ; **ORGANIZATION**, **LOCATION**, **NAME** và **USERNAME** dựa nhiều hơn vào ngữ cảnh nên được xếp sau các entity có cấu trúc rõ.

3.2.3 Thuật toán Greedy Priority-First

Thuật toán hợp nhất thực hiện giải quyết overlap theo chiến lược tham lam ưu tiên (greedy priority-first):

1. Gộp tất cả thực thể từ mọi detector thành một danh sách duy nhất.
2. Sắp xếp danh sách theo ba tiêu chí lần lượt: (a) thứ hạng ưu tiên tăng dần (loại ưu tiên cao xếp trước), (b) chỉ số **start** tăng dần, (c) độ dài span giảm dần (span dài hơn xếp trước khi cùng điểm xuất phát).
3. Duyệt tuần tự qua danh sách đã sắp xếp. Với mỗi thực thể, giữ lại nếu span của nó không chồng lấn với bất kỳ thực thể đã giữ nào; bỏ qua nếu ngược lại.
4. Sắp xếp lại danh sách kết quả theo **start** để đảm bảo thứ tự xuất hiện trong văn bản.

Algorithm 3 Thuật toán hợp nhất và giải quyết overlap (Greedy Priority-First)

Require: Các nhóm thực thể $G_1, G_2, \dots, G_k$ từ các detector

Ensure: Danh sách thực thể không overlap, sắp theo vị trí

```
1:  $all \leftarrow G_1 \cup G_2 \cup \dots \cup G_k$ 
2: Sắp xếp  $all$  theo ( $priority(type)$ ,  $start$ ,  $-(end - start)$ ) tăng dần
3:  $kept \leftarrow \emptyset$ 
4:  $occupied \leftarrow \emptyset$  // Tập các span  $(s, e)$  đã giữ
5: for mỗi thực thể  $ent$  trong  $all$  do
6:    $s \leftarrow ent.start$ ,  $e \leftarrow ent.end$ 
7:   if  $\nexists (os, oe) \in occupied$  sao cho  $s < oe$  và  $e > os$  then
8:      $kept \leftarrow kept \cup \{ent\}$ 
9:      $occupied \leftarrow occupied \cup \{(s, e)\}$ 
10:  end if
11: end for
12: Sắp xếp  $kept$  theo  $start$  tăng dần
13: return  $kept$ 
```

3.2.4 Minh họa trường hợp overlap

Xét đoạn văn bản sau:

"I live at 123 Main Street."

Trong đoạn này, ADDRESS và LOCATION có thể cùng nhận diện vùng văn bản liên quan đến Main Street. Danh sách trước khi hợp nhất có dạng như Bảng 3.1.

Bảng 3.1: Danh sách thực thể trước khi hợp nhất

Detector	Loại	Text	Ưu tiên
context_detector	ADDRESS	123 Main Street	4
context_detector	LOCATION	Main Street	6

Sau khi sắp xếp theo ưu tiên và áp dụng thuật toán greedy, merger giữ 123 Main Street vì ADDRESS có priority cao hơn LOCATION. Candidate Main Street bị loại do overlap với span địa chỉ đầy đủ đã được giữ. Kết quả cuối cùng được trình bày trong Bảng 3.2.

Bảng 3.2: Danh sách thực thể sau khi hợp nhất

Loại	Text	Vị trí
ADDRESS	123 Main Street	start=10, end=25

Kết quả này sau đó được đưa vào module **anonymizer** để thay thế các span PII bằng placeholder tương ứng, tạo ra văn bản đã được ẩn danh hóa.